



Xarxes neuronals i *deep learning*

Lluís A. Belanche

Computer Science Department
Universitat Politècnica de Catalunya
belanche@cs.upc.edu

LECTURE 2: General concepts of linear models

Version of February 24, 2025

2. General concepts of linear models

Part 1

- ① General introduction to the linear model.
- ② Classical regression analysis. A worked out example.
- ③ Extension of the model. Basis functions. Error functions.
- ④ Concept of regularization. Regularization by the norm. Ridge and LASSO regression.
- ⑤ Conclusions of Part 1.

Part 2

- ① Introduction to Generalized linear models (GLMs).
- ② The “data sample $\rightarrow$ algorithm” path.
- ③ Learning as an optimization problem.
- ④ Conclusions of Part 2.

2. General concepts of linear models

Part 1

- ① General introduction to the linear model.
- ② Classical regression analysis. A worked out example.
- ③ Extension of the model. Basis functions. Error functions.
- ④ Concept of regularization. Regularization by the norm. Ridge and LASSO regression.
- ⑤ Conclusions of Part 1.

Part 2

- ① Introduction to Generalized linear models (GLMs).
- ② The “data sample $\rightarrow$ algorithm” path.
- ③ Learning as an optimization problem.
- ④ Conclusions of Part 2.

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

We come from linear **regression** ... we liked it and ... we wish to extend it to other kinds of tasks (e.g., **classification**). How could we do this properly?

$$t_n | \mathbf{x}_n = \beta^\top \mathbf{x}_n + \beta_0$$

- 1 Gaussian $\rightarrow$???
- 2 identity $\rightarrow$???

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

We come from linear **regression** ... we liked it and ... we wish to extend it to other kinds of tasks (e.g., **classification**). How could we do this properly?

$$t_n | \mathbf{x}_n = \beta^\top \mathbf{x}_n + \beta_0$$

- 1 Gaussian $\rightarrow$???
- 2 identity $\rightarrow$???

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

We come from linear **regression** ... we liked it and ... we wish to extend it to other kinds of tasks (e.g., **classification**). How could we do this properly?

$$t_n | \mathbf{x}_n = \beta^\top \mathbf{x}_n + \beta_0$$

- ① Gaussian $\rightarrow$???
- ② identity $\rightarrow$???

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

We come from linear **regression** ... we liked it and ... we wish to extend it to other kinds of tasks (e.g., **classification**). How could we do this properly?

$$t_n | \mathbf{x}_n = \beta^\top \mathbf{x}_n + \beta_0$$

- ① Gaussian $\rightarrow$???
- ② identity $\rightarrow$???

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

We come from linear **regression** ... we liked it and ... we wish to extend it to other kinds of tasks (e.g., **classification**). How could we do this properly?

$$t_n | \mathbf{x}_n = \boldsymbol{\beta}^\top \mathbf{x}_n + \beta_0$$

- ① Gaussian $\rightarrow$???
- ② identity $\rightarrow$???

Generalized Linear Models:

- Very general and useful classical technique for fitting **linear models**.
- Work for many **target types**:
 - binary (two-class) and nominal (multi-class)
 - proportions and counts
 - ordinal (ordered classes)
 - continuous
 - ...
- Admit **general predictors** (categorical ones are automatically handled).
- Offer a “data sample $\longrightarrow$ algorithm” path.

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

Generalized Linear Model:

- A GLM is a linear predictor of a “convenient” function of the **conditional expectation** of the target variable:

$$h(\mathbb{E}[t_n|\mathbf{x}_n]) = \boldsymbol{\beta}^\top \mathbf{x}_n + \beta_0$$

- This convenient function h is typically smooth and invertible and called the **link function**.
- The t_n are taken as independent and drawn from a distribution of the **exponential family** (Poisson, Gaussian, Bernoulli, Gamma, ...).
- No distributional assumptions on the $\mathbf{x}$ data.

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

The modeller chooses a suitable distribution for t_n given $\mathbf{x}_n$:

- ① Gaussian distribution (h is identity): **linear regression**
 - ② Bernoulli distribution (h is logit): **logistic regression**
 - ③ Poisson distribution (h is $\ln$): **Poisson regression**
 - ④ $\vdots$
- This generality comes at a cost: in general we need an iterative procedure for the optimization of the β and β_0 parameters.
 - A popular procedure is to set it up as a MaxLik problem and use a preferred numerical optimization method (e.g. Newton-Raphson).

NEURAL NETWORKS AND DEEP LEARNING

GLM example: Logistic regression

We are in the case of binary classification (two classes or $K = 2$); we **model** the posterior probability for class ω_1 as:

$$y(\mathbf{x}_n) = P(\omega_1 | \mathbf{x}_n)$$

The idea is that the distribution is the Bernoulli:

$$t_n | \mathbf{x}_n \sim \text{Ber}(p_n)$$

and

$$h(y(\mathbf{x}_n)) = h(p_n) = \boldsymbol{\beta}^\top \mathbf{x}_n + \beta_0$$

This is so because if $Z \sim \text{Ber}(p)$, then $\mathbb{E}[Z] = p$.

note $P(\omega_2 | \mathbf{x}) = 1 - P(\omega_1 | \mathbf{x}) = 1 - y(\mathbf{x})$; define $h^{-1}(z) =: g(z)$.

NEURAL NETWORKS AND DEEP LEARNING

GLM example: Logistic regression

- In essence, $y(\mathbf{x}_n) = p_n = P(\omega_1 | \mathbf{x}_n)$ is what our model aims at.
- The specific choice we make for h is:

$$h(z) = \ln \left(\frac{z}{1-z} \right), \quad z \in (0, 1)$$

known as the **logit function**.

- Therefore

$$y(\mathbf{x}_n) = \frac{1}{1 + \exp(-\boldsymbol{\beta}^\top \mathbf{x}_n - \beta_0)}$$

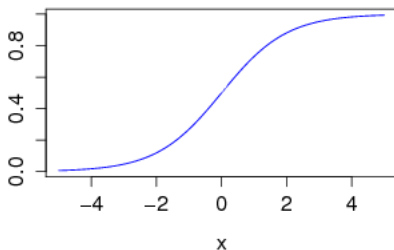
NEURAL NETWORKS AND DEEP LEARNING

GLM example: Logistic regression

The **logistic** function is a C^∞ function $g : \mathbb{R} \rightarrow (0, 1)$ with

$$g(z) = \frac{1}{1 + e^{-z}}.$$

The logistic function



This is a bijection (one-to-one), with inverse (our *link*) function!

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression: let's go for the parameters!

- Suppose we have an i.i.d. sample of N labelled observations $\mathcal{D}_n = \{(\mathbf{x}_n, t_n)\}$, $n = 1, \dots, N$, where $\mathbf{x}_n \in \mathbb{R}^d$, $t_n \in \{0, 1\}$.
- Let's re-write as usual $\mathbf{x}_n := (1, x_{n,1}, \dots, x_{n,d})^\top$ and $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_d)^\top$ so we write $P(\omega_1|\mathbf{x}) = g(\boldsymbol{\beta}^\top \mathbf{x})$.
- ① As for linear regression, we set the problem as a **Maximum Likelihood** problem with parameter vector $\boldsymbol{\beta}$ (which now includes β_0).
- ② Unlike linear regression, there is no closed-form solution and we must use an iterative **Newton-Raphson** method.
- ③ This leads to an iterative reestimation procedure for the $\boldsymbol{\beta}$ parameters (IRLS) $\longrightarrow \hat{\boldsymbol{\beta}}$.

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression: let's go for the parameters!

Each $t_n \sim \text{Ber}(p_n)$, where $p_n = g(\beta^\top \mathbf{x}_n)$,

$$P(t_n | \mathbf{x}_n, \beta) = \begin{cases} p_n & \text{if } t_n = 1 \ (\mathbf{x}_n \in \omega_1) \\ 1 - p_n & \text{if } t_n = 0 \ (\mathbf{x}_n \in \omega_2) \end{cases}$$
$$= p_n^{t_n} (1 - p_n)^{(1-t_n)}$$

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression: let's go for the parameters!

$$\begin{aligned}l(\beta) &= \ln \mathcal{L}(\beta) \\&= \ln P(\{t_n\}_{n=1}^N | \{\mathbf{x}_n\}_{n=1}^N, \beta) \\&= \ln \prod_{n=1}^N P(t_n | \mathbf{x}_n, \beta) \\&= \sum_{n=1}^N \ln P(t_n | \mathbf{x}_n, \beta) \\&= \sum_{n=1}^N \ln \left(g(\beta^\top \mathbf{x}_n)^{t_n} (1 - g(\beta^\top \mathbf{x}_n))^{(1-t_n)} \right) \\&= \sum_{n=1}^N \ln \left((p_n)^{t_n} (1 - p_n)^{(1-t_n)} \right), \quad p_n = g(\beta^\top \mathbf{x}_n)\end{aligned}$$

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression: let's go for the parameters!

Now

$$\begin{aligned}(p_n)^{t_n}(1-p_n)^{(1-t_n)} &= \left(\frac{p_n}{1-p_n}\right)^{t_n} (1-p_n) \\ &= \left(\exp(\boldsymbol{\beta}^\top \mathbf{x}_n)\right)^{t_n} \left(1 + \exp(\boldsymbol{\beta}^\top \mathbf{x}_n)\right)^{-1}\end{aligned}$$

Therefore

$$l(\boldsymbol{\beta}) = \sum_{n=1}^N \left[t_n \boldsymbol{\beta}^\top \mathbf{x}_n - \ln \left(1 + \exp(\boldsymbol{\beta}^\top \mathbf{x}_n) \right) \right]$$

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression: let's go for the parameters!

The MaxLik setup of the logistic regression does not have a closed-form solution:

$$\begin{aligned}\beta^{k+1} &= \beta^k - \left(\frac{\partial^2 l}{\partial \beta \partial \beta^\top} \right)^{-T} \left(\frac{\partial l}{\partial \beta} \right) \\ &= \beta^k + (\mathbf{X}^\top \mathbf{P} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{t} - \mathbf{p}) \\ &= (\mathbf{X}^\top \mathbf{P} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{P} \mathbf{z}, \quad \mathbf{z} := \mathbf{X} \beta^\top + \mathbf{P}^{-1}(\mathbf{t} - \mathbf{p})\end{aligned}$$

$\mathbf{X}$ is the matrix of the $\{\mathbf{x}_n\}$

$\mathbf{t} = (t_1, \dots, t_n)^\top$, $\mathbf{p} = (p_1, \dots, p_N)^\top$

and $\mathbf{P} = \text{diag}(p_n(1 - p_n))$, $n = 1, \dots, N$

$$\frac{\partial l}{\partial \beta} = \mathbf{X}^\top (\mathbf{t} - \mathbf{p})$$

$$\frac{\partial^2 l}{\partial \beta \partial \beta^\top} = -\mathbf{X}^\top \mathbf{P} \mathbf{X}$$

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression: interpretation (I): mantras

“The log of the odds is a linear function of the predictors”

Since $P(\omega_1|\mathbf{x}) = g(\boldsymbol{\beta}^\top \mathbf{x} + \beta_0)$ we have

$$\ln \left(\frac{P(\omega_1|\mathbf{x})}{P(\omega_2|\mathbf{x})} \right) = \ln \left(\frac{P(\omega_1|\mathbf{x})}{1 - P(\omega_1|\mathbf{x})} \right) = \text{logit}(P(\omega_1|\mathbf{x})) = \boldsymbol{\beta}^\top \mathbf{x} + \beta_0$$

“The function that relates the linear predictor to the mean of the predicted random variable is called the *link* function”

Homework

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression interpretation (II): the coefficients

$$\text{LOGODDS}(\mathbf{x}) = \ln \left(\frac{P(\omega_1|\mathbf{x})}{P(\omega_2|\mathbf{x})} \right) = \boldsymbol{\beta}^\top \mathbf{x} + \beta_0$$

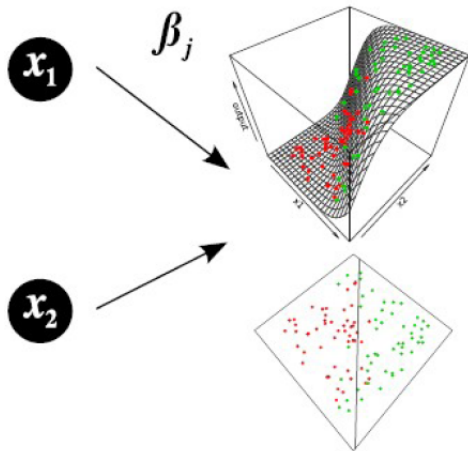
$$\text{ODDS}(\mathbf{x}) = \frac{P(\omega_1|\mathbf{x})}{P(\omega_2|\mathbf{x})} = \exp(\boldsymbol{\beta}^\top \mathbf{x} + \beta_0)$$

Define $\mathbf{1}_i := (0, \dots, \overset{i)}{1}, \dots, 0)^\top$, so
 $\mathbf{x} + \mathbf{1}_i = (x_{01}, \dots, x_{0i} + 1, \dots, x_{0N})^\top$

$$\frac{\text{ODDS}(\mathbf{x} + \mathbf{1}_i)}{\text{ODDS}(\mathbf{x})} = \exp((\boldsymbol{\beta}^\top (\mathbf{x} + \mathbf{1}_i - \mathbf{x})) = \exp(\beta_i)$$

NEURAL NETWORKS AND DEEP LEARNING

Logistic regression interpretation (and III): graphical view



NEURAL NETWORKS AND DEEP LEARNING

Logistic regression: the Deviance and the AIC

In the context of Generalized Linear Models,

$$-2l(\hat{\beta}) = -2 \ln \mathcal{L}(\hat{\beta})$$

is called the **deviance** (in ML, this is the **error**)

Null deviance: deviance of the null model (just with constant term)

Residual deviance: deviance of the proposed model

AIC

The AIC complements the deviance with complexity penalization
 $-2l(\hat{\beta}) + 2d$ (a form of **regularization**)

In many statistical studies, one tries to relate a count to some scientific variables:

- 1 Number of cardio-vascular accidents among people over 60 in a US state $\sim$ average income in the state
- 2 Number of bicycles in a Danish household $\sim$ distance to the city centre
- 3 Number of incoming calls to a complaints number $\sim$ hourly interval

When the response is a count which does not have any natural upper bound, the logistic regression is not appropriate.

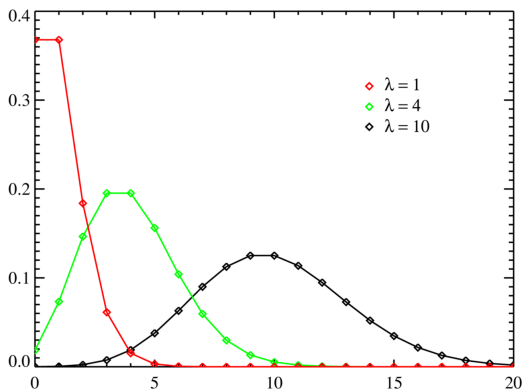
→ The **Poisson regression** is a natural alternative!

$\sim$ stands for “as a function of”

NEURAL NETWORKS AND DEEP LEARNING

Poisson regression

The Poisson is a discrete distribution $Z \sim \text{Pois}(\lambda)$ with probability mass function:



$$P(Z = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad \lambda > 0 \in \mathbb{R}, k = 0, 1, 2, \dots$$

NEURAL NETWORKS AND DEEP LEARNING

Poisson regression

- We consider independent Poisson random variables $t_1, \dots, t_n$ with $t_n \sim \text{Pois}(\lambda_n)$. We know that $\mathbb{E}[t_n] = \lambda_n$.
- We have an i.i.d. sample of N observations $\mathbf{x}_n \in \mathbb{R}^d$ (e.g., distance to the city centre).
- We have a corresponding sample $t_1, \dots, t_n$, where each t_n is drawn from t_n (e.g., number of bicycles)
- The idea is to model λ_n as $\exp(\beta^T \mathbf{x}_n + \beta_0)$ (the link function is $\ln$).
- The Poisson regression model is $t_n \sim \text{Pois}(\exp(\beta^T \mathbf{x}_n + \beta_0))$ or $\ln \lambda_n = \beta^T \mathbf{x}_n + \beta_0$.
- Proceeding like the previous case for LogReg, we arrive at an expression the optimization of which has no closed-form solution; however, we can still use N-R (because it is convex).

NEURAL NETWORKS AND DEEP LEARNING

Introduction to Generalized linear models (GLMs)

The “data sample $\rightarrow$ algorithm” path

- 1 Get an i.i.d data sample $\mathcal{D}_n$
- 2 Identify the target variables t_n
- 3 Choose a convenient distribution for $t_n|\mathbf{x}_n$ from the exponential family
- 4 Choose a convenient link function h and obtain the model
$$h(\mathbb{E}[t_n|\mathbf{x}_n]) = \boldsymbol{\beta}^\top \mathbf{x}_n + \beta_0$$
- 5 Build the log-likelihood function and set it up as a minimization problem
- 6 Derive an updating formula for the $\boldsymbol{\beta}$ and β_0 parameters via N-R
- 7 Embed the updating formula into an algorithm

Optimization view:

$$\min_{\theta \in \Theta} e_{\text{emp}}^{\lambda}(\theta) := \frac{1}{n} \sum_{n=1}^N L(t_n, f(\mathbf{x}_n; \theta)) + \lambda \Omega(f), \quad \lambda > 0$$

where:

- L is a loss/error function
- Ω is a regularizer function
- $\Omega \subseteq \mathbb{R}^k$ is the parameter space
- λ is the regularization constant (a hyper-parameter)

NEURAL NETWORKS AND DEEP LEARNING

Conclusions of Part 2

We have introduced **generalized linear models** as a technique capable of building models for many tasks.

ADVANTAGES

- 1 They can be further extended by the use of non-linear **basis functions**.
- 2 They can be further extended by adding a regularization term.
- 3 The computations are reasonably efficient.
- 4 Interpretability of the models is easy and well-founded.

DISADVANTAGES

- 1 Basically the same as for linear regression in Part 1!

—→ To leap forward, we need to add **data-dependent basis functions** to the fitting algorithms, and adapt everything else (training procedure, regularizer, ...) as well as understand better what kind of problems can be solved.

“It has been said that ‘*All models are wrong but some models are useful.*’ In other words, any model is at best a useful fiction —there never was, or ever will be, an exactly normal distribution or an exact linear relationship. Nevertheless, enormous progress has been made by entertaining such fictions and using them as approximations.”

— George Box (quoting himself).

NEURAL NETWORKS AND DEEP LEARNING

Relevant characters for this lecture



John Nelder
(1924-2010)



Robert Wedderburn
(1947-1975)



Simeon Poisson
(1781-1840)

End of Part 2